



comprehensive class note on "Programming in T-SQL",

- Definitions of terms
- Explanations
- Use cases
- Examples
- Classwork exercises
- Across three complexity levels: Basic, Intermediate, Advanced

🔧 Tool: SSMS

🗄 Database: AdventureWorks2022

💡 Class Note: Programming in T-SQL (Transact-SQL)

Focus: Writing reusable, logical, procedural SQL code for data operations using T-SQL.

📌 I. Definition of Key Terms

Term	Definition
T-SQL	Transact-SQL is Microsoft's extension of SQL, adding procedural programming, variables, loops, and error handling.
Batch	A group of SQL statements sent to SQL Server for execution at once.
Control-of-Flow	Statements like IF...ELSE, WHILE, GOTO, etc., used to control program logic.
Stored Procedure	A saved group of T-SQL statements that perform a task.

Term	Definition
Function	A T-SQL block that returns a value and can be used in queries.
Variables	Temporary storage of data used in T-SQL programs (<code>DECLARE</code> , <code>SET</code>).
TRY...CATCH	A block to handle runtime errors in T-SQL code.

🎯 Level 1: Basic T-SQL Programming

✓ Explanation

T-SQL supports procedural logic like declaring variables, making decisions, and repeating tasks.

✓ Use Case

- Set and manipulate variables
- Run simple conditional checks
- Basic printing and flow control

✓ Example 1: Declare & Set Variable

```
DECLARE @Name NVARCHAR(50);  
SET @Name = 'Joel';  
  
PRINT 'Hello ' + @Name;
```

✓ Example 2: IF...ELSE

```
DECLARE @Count INT = 5;  
  
IF @Count > 0  
    PRINT 'Positive number';  
ELSE  
    PRINT 'Zero or negative';
```

🔍📊🎯 Classwork 1 (Basic)

1. Declare a variable `@age` , set it to 22.
 2. Use `IF...ELSE` to print:
 - “Minor” if under 18
 - “Adult” if 18 or above
-

🎯 Level 2: Intermediate T-SQL Programming

✓ Explanation

Add loops, error handling, and multi-statement logic.

✓ Use Case

- Repeatedly insert or update data
- Execute actions conditionally
- Handle user input and errors

✓ Example: WHILE Loop

```
DECLARE @i INT = 1;

WHILE @i <= 5
BEGIN
    PRINT 'Count: ' + CAST(@i AS NVARCHAR);
    SET @i = @i + 1;
END
```

✓ Example: TRY...CATCH

```
BEGIN TRY
    DECLARE @x INT = 1, @y INT = 0;
    DECLARE @z INT;

    SET @z = @x / @y;
END TRY
BEGIN CATCH
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH
```

🔍📊🎯 Classwork 2 (Intermediate)

1. Use a `WHILE` loop to print the numbers 10 down to 1.
 2. Use `TRY...CATCH` to divide two numbers entered as variables. If division by zero occurs, catch and print an error.
-

● Level 3: Advanced T-SQL Programming

✓ Explanation

You can package logic into **stored procedures**, **functions**, and apply **modular logic**.

✓ Use Case

- Modular programming
 - Reuse of logic
 - Secure and optimized execution
-

✓ Example: Stored Procedure

```
CREATE PROCEDURE GetEmployeeByJobTitle
    @JobTitle NVARCHAR(50)
AS
BEGIN
    SELECT FirstName, LastName, JobTitle
    FROM HumanResources.Employee AS e
    JOIN Person.Person AS p ON e.BusinessEntityID = p.BusinessEntityID
    WHERE JobTitle = @JobTitle;
END;

-- Execute it
EXEC GetEmployeeByJobTitle 'Design Engineer';
```

✓ Example: Scalar Function

```
CREATE FUNCTION dbo.GetFullName(@First NVARCHAR(50), @Last NVARCHAR(50))
RETURNS NVARCHAR(100)
AS
BEGIN
    RETURN @First + ' ' + @Last;
END;

-- Usage
SELECT dbo.GetFullName('Joel', 'Odufu');
```

✓ Example: Table-Valued Function

```
CREATE FUNCTION dbo.GetProductsAbovePrice(@MinPrice MONEY)
RETURNS TABLE
AS
RETURN (
    SELECT ProductID, Name, ListPrice
    FROM Production.Product
    WHERE ListPrice > @MinPrice
);
```

```
-- Usage
SELECT * FROM dbo.GetProductsAbovePrice(100);
```

🔍📊🔧 Classwork 3 (Advanced)

- Write a stored procedure that:
 - Accepts a product ID
 - Returns the product name and price
 - Uses error handling
- Create a function `dbo.IsExpensiveProduct(@Price MONEY)` that returns `'Yes'` if price > 1000, else `'No'`.
- Create a table-valued function that returns employees by department name.

🔍 Summary

Feature	Purpose
DECLARE & SET	Store and assign values to variables
IF...ELSE / CASE	Conditional execution
WHILE	Loops
TRY...CATCH	Error handling

Feature	Purpose
Stored Procedures	Reusable execution blocks
Functions	Return values or tables; useful in SELECTs

💡 Suggested Practice

🔄 Assignments:

1. Write a loop that inserts 10 test products into a table with name `TestProduct_1` , `TestProduct_2` , etc.
 2. Create a stored procedure that returns employees hired after a certain year.
 3. Create a scalar function to return age based on date of birth.
-